

Programação Lógica

Unificação

Prof. Edson Alves

Campus UnB Gama: Faculdade de Ciências e Tecnologias em Engenharia

Processos de unificação

Processos de unificação

Elementos	Comportamento
-----------	---------------

Processos de unificação

Elementos	Comportamento
Variável e termo	Uma variável unifica com qualquer termo, inclusive outras variáveis. O valor do termo unificado é atado à variável em questão

Processos de unificação

Elementos	Comportamento
Variável e termo	Uma variável unifica com qualquer termo, inclusive outras variáveis. O valor do termo unificado é atado à variável em questão
Primitivo e primitivo	Dois termos primitivos (átomos ou números) unificam apenas se são são idênticos

Processos de unificação

Elementos	Comportamento
Variável e termo	Uma variável unifica com qualquer termo, inclusive outras variáveis. O valor do termo unificado é atado à variável em questão
Primitivo e primitivo	Dois termos primitivos (átomos ou números) unificam apenas se são idênticos
Estrutura e estrutura	Unificam se tem mesmo construtor, mesma aridade e cada par de argumentos correspondentes unificam

Predicado =/2

Predicado $=/2$

★ O predicado pré-definido $=/2$ é bem sucedido se ambos argumentos unificam, e falha, caso contrário

Predicado `=/2`

★ O predicado pré-definido `=/2` é bem sucedido se ambos argumentos unificam, e falha, caso contrário

```
?- a = a.  
true.
```

```
?- a = b.  
false.
```

```
?- x(y, z(i, j)) = x(y, z(i, j)).  
true.
```

```
?- x(y, z(i, j)) = x(y, z(j, i)).  
false.
```

```
? 2 + 2 = 4.  
false.
```

Unificação de variáveis e termos primitivos

Unificação de variáveis e termos primitivos

- ★ Ao unificar com um termo primitivo, a variável é atada ao seu valor

Unificação de variáveis e termos primitivos

- ★ Ao unificar com um termo primitivo, a variável é atada ao seu valor

?- $X = a$.

$X = a$.

?- $2 = Y$.

$Y = 2$.

?- $f(x, y) = f(X, y)$.

$X = x$.

?- $f(x, y) = f(X, z)$.

false.

?- $f(x, y) = f(X, Y)$.

$X = x$.

$Y = y$.

Unificação entre variáveis

Unificação entre variáveis

- ★ Duas variáveis sempre unificam

Unificação entre variáveis

★ Duas variáveis sempre unificam

?- A = B.

A = B.

?- f(X, y) = f(Z, y).

X = Z.

?- X = Y, Y = teste.

X = Y, Y = teste.

?- X = Y, a(Z) = a(Y), X = teste.

X = Y, Y = Z, Z = teste.

?- X = Y, Y = 2, write(X).

2

X = Y, Y = 2.

Unificação em consultas compostas

Unificação em consultas compostas

- ★ A variável anônima ‘_’ é um *wildcard* e não é atada a valor algum

Unificação em consultas compostas

- ★ A variável anônima ' ' é um *wildcard* e não é atada a valor algum
- ★ O predicado `\=/2` retorna verdadeiro se os seus argumentos não unificam

Unificação em consultas compostas

- ★ A variável anônima '_' é um *wildcard* e não é atada a valor algum
- ★ O predicado `\=/2` retorna verdadeiro se os seus argumentos não unificam

```
?- f(A, B) = C, write(C), nl, A = a, B = 8.
```

```
f(_6142, _6144)
```

```
A = a,
```

```
B = 8,
```

```
C = f(a, 8).
```

```
?- f(X) \= f(x, y).
```

```
true.
```

```
?- f(x, y, z) = f(_, _, z).
```

```
true.
```

```
male(homer).
male(bart).
female(marge).
female(lisa).
female(maggie).
father(homer, bart).
father(homer, lisa).
father(homer, maggie).
mother(marge, bart).
mother(marge, lisa).
mother(marge, maggie).
siblings(X, Y) :- X \= Y, father(F, X), father(F, Y).
brothers(X, Y) :- male(Y), siblings(X, Y).
sisters(X, Y) :- female(Y), siblings(X, Y).
```

% Exemplos de consultas

```
?- sisters(bart, S).
```

```
?- siblings(maggie, S).      % Explique o resultado desta consulta!
```

Listas em Prolog

Listas em Prolog

★ Em Prolog, uma **lista** é uma coleção de termos de quaisquer tipos. A sintaxe para declaração de uma lista é:

```
[term1, term2, ..., termN]
```

Listas em Prolog

★ Em Prolog, uma **lista** é uma coleção de termos de quaisquer tipos. A sintaxe para declaração de uma lista é:

`[term1, term2, ..., termN]`

★ Uma lista vazia, também denominada `nil`, é representada por `[]`

Listas em Prolog

★ Em Prolog, uma **lista** é uma coleção de termos de quaisquer tipos. A sintaxe para declaração de uma lista é:

`[term1, term2, ..., termN]`

★ Uma lista vazia, também denominada **nil**, é representada por `[]`

★ A unificação trata as listas como uma estrutura de dados

```
?- asserta(f([a, b, c], d)).
```

```
true.
```

```
?- f(X, d).
```

```
X = [a, b, c].
```

```
?- [_ , X, _] = [1, 2, 3].
```

```
X = 2.
```


Casamento de padrão em listas

Casamento de padrão em listas

★ A notação $[X \mid Y]$ ata X ao primeiro elemento da lista, chamado *head*, e ata Y a uma lista com todos os demais elementos, denominada *tail*

Casamento de padrão em listas

★ A notação $[X \mid Y]$ ata X ao primeiro elemento da lista, chamado *head*, e ata Y a uma lista com todos os demais elementos, denominada *tail*

```
?- [H|T] = [1, 2, 3, 4, 5], [A | B] = [6].
```

```
H = 1,
```

```
T = [2, 3, 4, 5],
```

```
A = 6,
```

```
B = [].
```

```
?- [H|T] = [].
```

```
false.
```

*% É possível listar vários elementos antes da barra, separados por
% vírgulas, mas após a barra pode haver uma única variável*

```
?- [A, B | T] = [1, 2, 3, 4, 5].
```

```
A = 1,
```

```
B = 2,
```

```
T = [3, 4, 5].
```

Listas e predicados

Listas e predicados

★ Embora tenha uma sintaxe especial, uma lista é, de fato, um predicado de dois argumentos

Listas e predicados

- ★ Embora tenha uma sintaxe especial, uma lista é, de fato, um predicado de dois argumentos
- ★ O primeiro argumento do predicado `'[]'`/2 é o *head* da lista e o segundo é o *tail*

```
?- L = '[]'(1, '[]'(2, '[]'(3, []))).  
L = [1, 2, 3].
```

Listas e predicados

- ★ Embora tenha uma sintaxe especial, uma lista é, de fato, um predicado de dois argumentos
- ★ O primeiro argumento do predicado `'[]'`/2 é o *head* da lista e o segundo é o *tail*

```
?- L = '[]'(1, '[]'(2, '[]'(3, []))).  
L = [1, 2, 3].
```

- ★ A estrutura da lista é adequada para rotinas recursivas:

```
% Uma implementação possível para o predicado length/2, que computa  
% número de elementos em uma lista  
length_([_|T], X) :- length_(T, NewX), X is NewX + 1.  
length_([], 0).
```

% L é uma lista com os N primeiros números primos

% Complexidade: $O(N \wedge 3/2)$

```
primes(N, L) :- primes_(N, 2, L).
```

```
primes_(N, X, L) :-
```

```
    N > 0,
```

```
    (is_prime(X) -> succ(NewN, N) ; NewN = N),
```

```
    succ(X, NewX),
```

```
    primes_(NewN, NewX, NewL),
```

```
    (NewN \= N -> L = [X | NewL] ; L = NewL).
```

```
primes_(0, _, []).
```


% Determina se o número X é ou não primo

`is_prime(X) :- is_prime_(X, 3).`

`is_prime(2).`

`is_prime_(X, D) :-`

`X > 2,`

`mod(X, 2) =:= 1,`

`S is D ^ 2,`

`(S =< X`

`-> (mod(X, D) =\= 0,`

`NewD is D + 2,`

`is_prime_(X, NewD))`

`; true).`

Teste de pertinência

Teste de pertinência

- ★ O predicado `member/2` determina se um termo é ou não membro da lista indicada

Teste de pertinência

- ★ O predicado `member/2` determina se um termo é ou não membro da lista indicada
- ★ Ele pode implementado da seguinte maneira:

```
member_(H, [H|_]).  
member_(X, [_|T]) :- member_(X, T).
```

Teste de pertinência

- ★ O predicado `member/2` determina se um termo é ou não membro da lista indicada
- ★ Ele pode implementado da seguinte maneira:

```
member_(H, [H|_]).  
member_(X, [_|T]) :- member_(X, T).
```

- ★ A primeira cláusula verifica se o elemento é o primeiro elemento (*head*) da lista

Teste de pertinência

- ★ O predicado `member/2` determina se um termo é ou não membro da lista indicada
- ★ Ele pode implementado da seguinte maneira:

```
member_(H, [H|_]).  
member_(X, [_|T]) :- member_(X, T).
```

- ★ A primeira cláusula verifica se o elemento é o primeiro elemento (*head*) da lista
- ★ A segunda é a cláusula recursiva, que procura o elemento no restante (*tail*) da lista

Teste de pertinência

- ★ O predicado `member/2` determina se um termo é ou não membro da lista indicada
- ★ Ele pode implementado da seguinte maneira:

```
member_(H, [H|_]).  
member_(X, [_|T]) :- member_(X, T).
```

- ★ A primeira cláusula verifica se o elemento é o primeiro elemento (*head*) da lista
- ★ A segunda é a cláusula recursiva, que procura o elemento no restante (*tail*) da lista
- ★ Ambas falham em listas vazias, pois `[]` e `[H|T]` não unificam

Concatenação de listas

Concatenação de listas

★ O predicado `append/3` anexa o segundo argumento ao primeiro, unificando o resultado com o terceiro:

```
?- append([1, 2], [3, 4, 5], X).  
X = [1, 2, 3, 4, 5].
```

Concatenação de listas

★ O predicado `append/3` anexa o segundo argumento ao primeiro, unificando o resultado com o terceiro:

```
?- append([1, 2], [3, 4, 5], X).  
X = [1, 2, 3, 4, 5].
```

★ Ele pode implementado da seguinte maneira:

```
append([], X, X).  
append([H|T1], X, [H|T2]) :-  
    append(T1, X, T2).
```

Concatenação de listas

★ O predicado `append/3` anexa o segundo argumento ao primeiro, unificando o resultado com o terceiro:

```
?- append([1, 2], [3, 4, 5], X).  
X = [1, 2, 3, 4, 5].
```

★ Ele pode implementado da seguinte maneira:

```
append([], X, X).  
append([H|T1], X, [H|T2]) :-  
    append(T1, X, T2).
```

★ `append/3` também pode ser utilizado para decompor listas

% Exemplo de decomposição de lista por meio do predicado append/3

?- append(X, Y, [1, 2, 3, 4]).

X = [],

Y = [1, 2, 3, 4] ;

X = [1],

Y = [2, 3, 4] ;

X = [1, 2],

Y = [3, 4] ;

X = [1, 2, 3],

Y = [4] ;

X = [1, 2, 3, 4],

Y = [] ;

false.

Listas e fatos

Listas e fatos

★ É possível declarar fatos a partir de uma lista, consultando `assertz/1` no *head* da lista recursivamente

```
list_to_facts([]).  
list_to_facts([H|T]) :-  
    assertz(H),  
    list_to_facts(T).
```

Listas e fatos

★ É possível declarar fatos a partir de uma lista, consultando `assertz/1` no *head* da lista recursivamente

```
list_to_facts([]).  
list_to_facts([H|T]) :-  
    assertz(H),  
    list_to_facts(T).
```

★ Para o processo inverso, Prolog disponibiliza o predicado `findall/3`

Listas e fatos

★ É possível declarar fatos a partir de uma lista, consultando `assertz/1` no *head* da lista recursivamente

```
list_to_facts([]).  
list_to_facts([H|T]) :-  
    assertz(H),  
    list_to_facts(T).
```

★ Para o processo inverso, Prolog disponibiliza o predicado `findall/3`

★ O primeiro argumento é o padrão para os termos, o segundo o objetivo e o terceiro é a lista resultante

% Exemplo de declaração de fatos a partir de uma lista.

```
?- list_to_facts([f(1), f(1, 2), g(3)]).
```

```
true.
```

```
?- f(1).
```

```
true.
```

```
?- f(1, 2).
```

```
true.
```

```
?- g(1).
```

```
false.
```

```
?- g(3).
```

```
true.
```

*% A lista resultante contém os valores da variável do primeiro parâmetro
% que satisfizeram o objetivo do segundo parâmetro*

```
?- findall(X, f(X) ; g(X), L).
```

```
L = [1, 3].
```

Predicado **maplist**/**N**

Predicado `maplist/N`

★ O predicado `maplist/N` retorna verdadeiro se o objetivo `Goal` é bem sucedido para todas $(N - 1)$ -uplas de elementos correspondentes das listas indicadas

Predicado **maplist**/**N**

★ O predicado **maplist**/**N** retorna verdadeiro se o objetivo **Goal** é bem sucedido para todas $(N - 1)$ -uplas de elementos correspondentes das listas indicadas

★ Ele pode ser definido da seguinte maneira:

```
maplist(G, [X_11, ..., X_1n],  
          [X_21, ..., X_2n],  
          ...,  
          [X_m1, ..., X_mn]) :-  
    call(G, X_11, ..., X_m1),  
    call(G, X_12, ..., X_m2),  
    ...,  
    call(G, X_1n, ..., X_mn).
```

Predicado `maplist/N`

★ O predicado `maplist/N` retorna verdadeiro se o objetivo `Goal` é bem sucedido para todas $(N - 1)$ -uplas de elementos correspondentes das listas indicadas

★ Ele pode ser definido da seguinte maneira:

```
maplist(G, [X_11, ..., X_1n],  
         [X_21, ..., X_2n],  
         ...,  
         [X_m1, ..., X_mn]) :-  
    call(G, X_11, ..., X_m1),  
    call(G, X_12, ..., X_m2),  
    ...,  
    call(G, X_1n, ..., X_mn).
```

★ SWI-Prolog disponibiliza versões deste predicado para $N \in [2, 5]$

```
% Determina se X é múltiplo de Y  
is_multiple(X, Y) :- X mod Y == 0.
```

```
% Atente à aplicação parcial do predicado is_multiple  
common_multiple(X, L) :- maplist(is_multiple(X), L).
```

```
% Verdadeiro se ao menos um, ou todos, os membros são verdadeiros, respectivamente  
any(L) :- member(X, L), call(X).  
all(L) :- maplist(call, L).
```

```
% Conversão de graus Celsius para Fahrenheit
```

```
celsius_fahrenheit(C, F) :-  
    ((var(C), var(F))  
     -> instantiation_error('Ao menos uma variável deve estar atada') ; true),  
    (nonvar(C) -> F is C*9/5 + 32 ; C is (F - 32)*5/9).
```

```
% Exemplo de chamada
```

```
?- maplist(celsius_fahrenheit, [-1, 100, 40], F).  
F = [30.2, 212, 104].
```

```
% Determina todos os subconjuntos de um conjunto de N elementos
% length/2 retorna verdadeiro se N é o tamanho da lista S
% between(A, B, C) retorna verdadeiro se C pertence ao intervalo [A, B]
subset(N, S) :-
    length(S, N),
    maplist(between(0, 1), S).

% Gera uma lista de tamanho N, com todos os elementos iguais a Value
replicate(N, Value, L) :-
    length(L, N),
    maplist(=(Value), L).

% Gera o histograma dos inteiros contidos na lista Ns
% msort/2 ordena os elementos do primeiro parâmetro, sem remover duplicatas
% clumped/2 gera o RLE (run-length encoding) do primeiro parâmetro
histogram(Ns, H) :-
    msort(Ns, Sorted),
    clumped(Sorted, H).
```

Predicado `include`/3

Predicado `include/3`

- ★ O predicado `include/3` filtra os elementos da lista para os quais o objetivo é verdadeiro

Predicado **include**/3

★ O predicado **include**/3 filtra os elementos da lista para os quais o objetivo é verdadeiro

★ A sintaxe é

```
include(Goal, List, Filtered).
```

Predicado **include**/3

★ O predicado **include**/3 filtra os elementos da lista para os quais o objetivo é verdadeiro

★ A sintaxe é

```
include(Goal, List, Filtered).
```

★ O predicado **exclude**/3 é similar, porém mantém os elementos para os quais o objetivo falha

Predicado **include**/3

- ★ O predicado **include**/3 filtra os elementos da lista para os quais o objetivo é verdadeiro

- ★ A sintaxe é

`include(Goal, List, Filtered).`

- ★ O predicado **exclude**/3 é similar, porém mantém os elementos para os quais o objetivo falha

- ★ Os parâmetros são os mesmos do predicado **include**/3, com mesma ordem

% Determina se X é ou não ímpar

`is_odd(X) :- X mod 2  $\neq$  0.`

% Determina os números ímpares presente na lista L

`odds(L, Odds) :- include(is_odd, L, Odds).`

% Determina os números pares presente na lista L

`evens(L, Evens) :- exclude(is_odd, L, Evens).`

% Exemplo de consulta

% O predicado numlist/3 gera uma lista com todos os inteiros no intervalo dado

`?- numlist(1, 5, L), odds(L, Odds), evens(L, Evens).`

`L = [1, 2, 3, 4, 5],`

`Odds = [1, 3, 5],`

`Evens = [2, 4].`

Predicado **foldl**/**N**

Predicado **foldl/N**

★ O predicado **foldl/N** “dobra” os elementos de $m \in [1, 4]$ listas por meio do objetivo **Goal/(m+1)**, onde V_0 é o valor inicial

Predicado **foldl**/N

★ O predicado **foldl**/N “dobra” os elementos de $m \in [1, 4]$ listas por meio do objetivo **Goal**/(**m**+1), onde V_0 é o valor inicial

★ Ele pode ser definido da seguinte maneira:

```
foldl(G, [X_11, ..., X_1n],  
      [X_21, ..., X_2n],  
      ...,  
      [X_m1, ..., X_mn], V0, V) :-  
  call(G, X_11, ..., X_m1, V0, V1),  
  call(G, X_12, ..., X_m2, V1, V2),  
  ...,  
  call(G, X_1n, ..., X_mn, V<n-1>, V).
```


Predicado **foldl**/N

★ O predicado **foldl**/N “dobra” os elementos de $m \in [1, 4]$ listas por meio do objetivo **Goal**/(**m**+1), onde V_0 é o valor inicial

★ Ele pode ser definido da seguinte maneira:

```
foldl(G, [X_11, ..., X_1n],  
      [X_21, ..., X_2n],  
      ...,  
      [X_m1, ..., X_mn], V0, V) :-  
  call(G, X_11, ..., X_m1, V0, V1),  
  call(G, X_12, ..., X_m2, V1, V2),  
  ...,  
  call(G, X_1n, ..., X_mn, V<n-1>, V).
```

★ O predicado **scanl**/N é similar, mantendo porém os resultados parciais

% Computa a soma de todos os inteiros no intervalo [A, B]

% O predicado plus(A, B, C) é verdadeiro se $C = A + B$

```
summation(A, B, Sum) :-  
    numlist(A, B, L),  
    foldl(plus, L, 0, Sum).
```

% Computa a soma dos prefixos da lista de inteiros L

% O valor inicial será o primeiro elemento da lista resultante

```
prefix_sum(L, Psum) :-  
    scanl(plus, L, 0, Psum).
```

*% Verdadeiro se $C = A * B$*

```
multiply(A, B, C) :- C is A * B.
```

% Computa o N-ésimo fatorial

```
factorial(N, Fact) :-  
    numlist(1, N, L),  
    foldl(multiply, L, 1, Fact).
```

Referências

- ★ **BLACKBURN**, Patrick, **BOS**, Johan, **STRIEGNITZ**, Kristina. *Learn Prolog Now!*, Online, 2026.
- ★ **MERRIT**, Dennis. *Adventure in Prolog*, Amzi!, 191 pgs, 2017.
- ★ **SWI-Prolog**. <https://www.swi-prolog.org/>, acesso em 13/03/2026.
- ★ **Wikipédia**. *Prolog*, acesso em 13/03/2026.